

# Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



## 2028\_NeoColab\_Competitive Coding II\_L8

### VTU\_Week 7\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

A publishing company is building a digital index system for storing book references. Each reference is represented by a single uppercase character. To ensure efficient organization, the system uses a Binary Search Tree (BST):

If the new character is smaller than the current node, it is placed in the left subtree. If the new character is greater, it is placed in the right subtree.

Once all references are stored in the BST, the system needs to perform a postorder traversal to generate a summary sequence, which is then used for indexing and cross-referencing.

Your task is to help the publishing company build the BST from the given characters and print the postorder traversal.

Example:

Input:

5

Z E W T Y

Output:

Postorder traversal: T Y W E Z

### ***Input Format***

The first line consists of an integer N, representing the number of characters.

The second line consists of N space-separated characters.

### ***Output Format***

The output displays the post-order traversal of the given inputs as space-separated.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 5

Z E W T Y

Output: Postorder traversal: T Y W E Z

### ***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
// Structure for the BST node
```

```
struct Node {
    char data;
    struct Node* left;
    struct Node* right;
};
```

// Function to create a new node

```
struct Node* createNode(char val) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = val;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode;  
}
```

// Function to insert a character into the BST

```
struct Node* insert(struct Node* root, char val) {  
    if (root == NULL) {  
        return createNode(val);  
    }  
  
    if (val < root->data) {  
        root->left = insert(root->left, val);  
    } else if (val > root->data) {  
        root->right = insert(root->right, val);  
    }  
  
    return root;  
}
```

// Function to perform Postorder Traversal (Left, Right, Root)

```
void postorder(struct Node* root) {  
    if (root == NULL) {  
        return;  
    }  
  
    postorder(root->left);  
    postorder(root->right);  
    printf("%c ", root->data);  
}
```

int main() {

```
    int n;  
    if (scanf("%d", &n) != 1) return 0;
```

```
    struct Node* root = NULL;
```

```
    for (int i = 0; i < n; i++) {
```

```

char val;
// The space before %c skips any whitespace/newlines
scanf(" %c", &val);
root = insert(root, val);
}

printf("Postorder traversal: ");
postorder(root);
printf("\n");

return 0;
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Thiru is learning about binary search trees and wants to practice creating and traversing them. He needs to write a program that takes a series of characters as input and constructs a binary search tree with those characters.

After constructing the tree, Thiru wants to perform an in-order traversal and print all the characters in sorted order.

### **Input Format**

The first line contains an integer  $n$ , representing the number of characters to be inserted into the binary search tree.

The second line contains  $n$  space-separated characters  $c_i$ , representing the characters to be inserted into the binary search tree.

### **Output Format**

The output prints a single line containing all the characters of the binary search tree in sorted order, obtained by performing an in-order traversal of the tree, separated by a space.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 4

r e f s

Output: e f r s

### **Answer**

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure for the BST node
```

```
struct Node {
```

```
    char data;
```

```
    struct Node* left;
```

```
    struct Node* right;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* createNode(char val) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = val;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to insert a character into the BST
```

```
struct Node* insert(struct Node* root, char val) {
```

```
    if (root == NULL) {
```

```
        return createNode(val);
```

```
    }
```

```
    if (val < root->data) {
```

```
        root->left = insert(root->left, val);
```

```
    } else if (val > root->data) {
```

```
        root->right = insert(root->right, val);
```

```
    }
```

```
    return root;
```

```
}
```

```
// In-order Traversal: Left -> Root -> Right
```

// This produces the characters in sorted order

```
void inorder(struct Node* root) {  
    if (root == NULL) {  
        return;  
    }  
    inorder(root->left);  
    printf("%c ", root->data);  
    inorder(root->right);  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;  
    struct Node* root = NULL;  
    for (int i = 0; i < n; i++) {  
        char val;  
        scanf(" %c", &val); // Space before %c is vital to skip whitespace  
        root = insert(root, val);  
    }  
  
    inorder(root);  
    printf("\n");  
  
    return 0;  
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Thilak is studying binary search trees (BSTs) and wants to implement a program that constructs a binary search tree from a given set of characters. He aims to perform a pre-order traversal of the constructed tree to understand its structure better.

Your task is to help Thilak by designing a program that accomplishes this.

#### **Input Format**

The first line of input consists of an integer N, representing the number of

characters to be inserted into the BST.

The second line consists of N space-separated characters, representing the data elements to be inserted into the BST.

### ***Output Format***

The output prints the pre-order traversal of the constructed BST.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 5

C A E B D

Output: C A B E D

### ***Answer***

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure for the BST node
```

```
struct Node {
```

```
    char data;
```

```
    struct Node* left;
```

```
    struct Node* right;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* createNode(char val) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = val;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to insert a character into the BST
```

```
struct Node* insert(struct Node* root, char val) {
```

```
    if (root == NULL) {
```

```

        return createNode(val);
    }

    if (val < root->data) {
        root->left = insert(root->left, val);
    } else if (val > root->data) {
        root->right = insert(root->right, val);
    }

    return root;
}

```

```

// Pre-order Traversal: Root -> Left -> Right
void preorder(struct Node* root) {
    if (root == NULL) {
        return;
    }

```

```

    // Process the root first
    printf("%c ", root->data);
    // Then recurse on left and right
    preorder(root->left);
    preorder(root->right);
}

```

```

int main() {
    int n;
    // Reading the number of elements
    if (scanf("%d", &n) != 1) return 0;

```

```

    struct Node* root = NULL;
    for (int i = 0; i < n; i++) {
        char val;
        // Space before %c skips leading whitespace/newlines
        scanf(" %c", &val);
        root = insert(root, val);
    }

```

```

    // Output the traversal
    preorder(root);
    printf("\n");
}

```



```
    return 0;  
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Jake is learning about binary search trees(BST) and their operations. He wants to implement a program that can delete a node from a BST based on the given key value and print the remaining nodes in an in-order traversal.

Assist Jake in the program.

##### **Input Format**

The first line of input consists of an integer n, representing the number of elements in BST.

The second line consists of n space-separated integers, representing the elements of the tree.

The third line consists of an integer x, representing the key value of the node to be deleted.

##### **Output Format**

The first line of output prints "Before deletion: " followed by the in-order traversal of the initial BST.

The second line prints "After deletion: " followed by the in-order traversal after the deletion of the key value.

If the key value is not present in the BST, print the original tree as it is.

Refer to the sample output for formatting specifications.

##### **Sample Test Case**

Input: 5

8 6 4 3 1

4

Output: Before deletion: 1 3 4 6 8

After deletion: 1 3 6 8

### **Answer**

// You are using GCC

#include <stdio.h>

#include <stdlib.h>

struct Node {

int data;

struct Node \*left, \*right;

};

struct Node\* createNode(int val) {

struct Node\* newNode = (struct Node\*)malloc(sizeof(struct Node));

newNode->data = val;

newNode->left = newNode->right = NULL;

return newNode;

}

struct Node\* insert(struct Node\* root, int val) {

if (root == NULL) return createNode(val);

if (val < root->data)

root->left = insert(root->left, val);

else if (val > root->data)

root->right = insert(root->right, val);

return root;

}

// Helper function to find the minimum value node in a subtree

struct Node\* minValueNode(struct Node\* node) {

struct Node\* current = node;

while (current && current->left != NULL)

current = current->left;

return current;

}

struct Node\* deleteNode(struct Node\* root, int key) {

if (root == NULL) return root;

```

// Standard BST search
if (key < root->data)
    root->left = deleteNode(root->left, key);
else if (key > root->data)
    root->right = deleteNode(root->right, key);
else {
    // Node with only one child or no child
    if (root->left == NULL) {
        struct Node* temp = root->right;
        free(root);
        return temp;
    } else if (root->right == NULL) {
        struct Node* temp = root->left;
        free(root);
        return temp;
    }

    // Node with two children: Get the inorder successor (smallest in the right
    subtree)
    struct Node* temp = minValueNode(root->right);
    root->data = temp->data;
    // Delete the inorder successor
    root->right = deleteNode(root->right, temp->data);
}
return root;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    int n, val, x;
    struct Node* root = NULL;

    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);

```

```

    root = insert(root, val);
}
scanf("%d", &x);

printf("Before deletion: ");
inorder(root);
printf("\n");

root = deleteNode(root, x);

printf("After deletion: ");
inorder(root);
printf("\n");

return 0;
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

You are required to implement basic operations on a Binary Search Tree (BST), like insertion and searching.

**Insertion:** Given a list of integers, construct a Binary Search Tree by repeatedly inserting each integer into the tree according to the rules of a BST.

**Searching:** Given an integer, search for its presence in the constructed Binary Search Tree. Print whether the integer is found or not.

Write a program to calculate this efficiently.

### ***Input Format***

The first line of input consists of an integer  $n$ , representing the number of nodes in the binary search tree.

The second line consists of the values of the nodes, separated by space as integers.

The third line consists of an integer representing, the value that is to be searched.

### **Output Format**

The output prints, "Value <value> is found in the tree." if the given value is present, otherwise it prints: "Value <value> is not found in the tree."

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 7

8 3 10 1 6 14 23

6

Output: Value 6 is found in the tree.

### **Answer**

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure for the BST node
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* left;
```

```
    struct Node* right;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int val) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = val;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to insert a value into the BST
```

```
struct Node* insert(struct Node* root, int val) {
```

```
    if (root == NULL) {
```

```
        return createNode(val);
```

```

    }
    if (val < root->data) {
        root->left = insert(root->left, val);
    } else if (val > root->data) {
        root->right = insert(root->right, val);
    }
    return root;
}

```

```

// Function to search for a value in the BST
struct Node* search(struct Node* root, int key) {
    // Base Case: root is null or key is present at root
    if (root == NULL || root->data == key) {
        return root;
    }

```

```

    // Key is greater than root's data
    if (key > root->data) {
        return search(root->right, key);
    }

```

```

    // Key is smaller than root's data
    return search(root->left, key);
}

```

```

int main() {
    int n, key;
    struct Node* root = NULL;

```

```

    // Read number of nodes
    if (scanf("%d", &n) != 1) return 0;

```

```

    // Read and insert node values
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        root = insert(root, val);
    }

```

```

    // Read the value to search for
    scanf("%d", &key);

```

```
// Perform search and print result
struct Node* result = search(root, key);
if (result != NULL) {
    printf("Value %d is found in the tree.\n", key);
} else {
    printf("Value %d is not found in the tree.\n", key);
}

return 0;
}
```

**Status :** Correct

**Marks :** 10/10